

Scaling Enterprise Agent Deployment: A Survey of LLM-Based Operational Grounding Construction

Jiaxin Bai¹, Lihui Liu², Yisen Gao⁴, Zihao Wang³, Yangqiu Song⁴

¹Hong Kong Baptist University ²Wayne State University

³TSY Capital ⁴Hong Kong University of Science and Technology

baijiaxin@hkbu.edu.hk hw6926@wayne.edu

zihao0710@gmail.com ygaodi@cse.ust.hk yqsong@cse.ust.hk

<https://github.com/HKBU-KnowComp/awesome-operational-grounding>

Abstract

Enterprise LLM agents still depend on engineers to hand-build the grounding layer that links schemas, policies, API documentation, and workflows to agent behavior. This survey studies LLM-based construction of that layer, which we call Automated Operational Grounding Construction (AOGC), framed as schema-level extraction and constrained structured generation of ontologies, validation rules, action and workflow models, API and tool contracts, policies, and maintenance records. We contribute a five-dimension requirements framework, a three-tier autonomy taxonomy, and a gap analysis that separates artifact coverage from enterprise-realistic validation. The literature has advanced individual pieces such as entity extraction, constraints, workflows, tools, and governance, but no coded method constructs and maintains a linked entity–constraint–action–governance–evolution layer from heterogeneous enterprise sources. We close with an agenda for co-constructing linked artifacts, formalizing governance with provenance, evaluating downstream agent behavior, and maintaining operational grounding over time.

1 Introduction

Enterprise LLM agents are moving from speculative demos to production deployment. Analyses emphasize generative AI’s productivity potential in knowledge work and enterprise knowledge management (McKinsey Global Institute, 2023), while protocols including the Model Context Protocol standardize how LLM applications connect to external data, tools, resources, and authorization-sensitive capabilities (Anthropic, 2024). Forward-deployed engineering has become a recurring enterprise-AI delivery pattern: Palantir’s SEC filings describe engineers embedded with users to surface product needs from field deployments, and OpenAI, Anthropic, ServiceNow, and Accenture describe

similar programs for bringing AI systems from pilots into production (Palantir Technologies, 2021; OpenAI, 2026; Anthropic, 2026; ServiceNow and Accenture, 2026). These signals point to a deployment bottleneck: organizations need current, permissioned, executable context, not only stronger base models or longer prompts.

The bottleneck is constructing the operational context that lets an agent act. An LLM-based insurance agent, for instance, must know more than the meanings of *claim*, *policy*, and *adjuster*: it must know that a claim has a claimant and an incident, that submitted claims require documentation, that high-value claims need senior review, that trainees may not approve payouts, and that new regulations may change documentation requirements after deployment. Comparable requirements appear in finance, procurement, healthcare, and customer operations, where agents must interpret domain state, select available actions, obey policies, and update behavior when the organization changes.

This knowledge is rarely stored as one neat ontology. Enterprise systems spread grounding knowledge across database schemas, API specifications, workflow engines, access-control tables, policy manuals, spreadsheets, and legacy documentation. We call the integrated deployed object the *operational grounding layer*. Its vocabulary core supplies classes, attributes, and relations; the wider layer links that vocabulary to constraints, action semantics, permissions, provenance, and versions. We use this term rather than “operational ontology” to avoid the implication that every API contract, workflow graph, or policy file is itself an ontology in the formal knowledge-representation sense.

Constructing this layer is still manual and expert-intensive. Engineers read heterogeneous sources, identify schema-level domain structure, translate requirements into validators or rules, connect actions to data objects, encode governance, and repair inconsistencies. We define *Automated Oper-*

Table 1: Five dimensions of operational grounding requirements. A single source sentence may map to different artifacts depending on whether the deployment needs inference, validation, action execution, policy enforcement, or maintenance.

Dim.	Function	Enterprise realizations	Research examples
D1	Entity and relationship structure for interpreting state	DB schemas, CRM objects, API types, GraphQL/Protobuf models	Vocabulary languages (OWL/RDFS), taxonomies, concept lattices
D2	Valid states, derivations, and violation checks	SQL constraints, JSON Schema, business rules, validation services	Validation rules (SHACL/ShEx), axioms, rule languages, reasoner checks
D3	Callable operations and process dynamics	OpenAPI, tool schemas, workflow DAGs, state machines, RPA scripts	Action models (PDDL), workflows (BPMN), process graphs, tool contracts
D4	Normative and organizational control over actions and data	RBAC/ABAC, OPA/Rego, retention policies, compliance workflows	Access-control and policy rules, deontic logic, privacy vocabularies
D5	Change, provenance, and consistency over time	Schema migrations, API versions, GitOps, feature flags	Ontology evolution, RDF deltas, belief revision, temporal KGs

ational Grounding Construction (AOGC) as the use of LLMs to construct, validate, or maintain these structured grounding artifacts. From an NLP perspective, AOGC is schema-level information extraction and constrained structured generation rather than standard instance-level KG population: the output must be parseable, internally coherent, grounded in evidence, and useful for downstream agent behavior. The central challenge is cross-artifact fidelity, because a correct schema fragment can still yield unsafe behavior if the corresponding action, permission, or version constraint is missing.

Nearby surveys and benchmarks motivate but do not cover this problem. [Garijo et al. \(2024\)](#) survey LLMs for ontology engineering, and [Bian \(2025\)](#) survey LLM-enabled KG construction; both are broader than enterprise agent grounding and do not organize the literature around action semantics, governance, and downstream behavior. LLMs4OL evaluates isolated ontology-learning subtasks ([Babaei Giglou et al., 2023](#)), and agent benchmarks such as SWE-bench, WebArena, GAIA, AgentBench, and τ -bench evaluate behavior in realistic software, web, assistant, interactive, and tool-use settings ([Jimenez et al., 2024](#); [Zhou et al., 2024a](#); [Mialon et al., 2024](#); [Liu et al., 2023b](#); [Yao et al., 2024](#)). AOGC studies the preceding construction problem: how to build and maintain the grounding layer that those agent environments assume is already available. Our autonomy framing follows recent ACL survey practice of making the role of LLMs explicit ([Zheng et al., 2025](#)), but retargets the axis to grounding-layer construction.

This survey makes three contributions. First, we present a function-first requirements framework for operational grounding layers that covers entities,

integrity rules, actions, governance, and maintenance, and treats ontology languages, validation rules, planning models, API contracts, workflow models, and policies as alternative structured outputs of the same underlying requirements. Second, we classify LLM-based methods into three autonomy tiers: subtask tools, fixed construction pipelines, and construction agents. Third, we distinguish *content gaps*, where a dimension is not yet seriously constructed, from *realism gaps*, where methods are evaluated under conditions too clean to support enterprise-readiness claims.

Scope and coding. We reviewed LLM-based work published or publicly available from January 2022 through 21 May 2026 using query families for ontology, schema, constraint, action, API/tool, policy, and maintenance construction. We retained papers that construct, validate, or maintain schema-, action-, rule-, policy-, or maintenance-level artifacts, including ontology languages, validation-rule languages, planning and action models, API specifications, workflows, policies, or comparable structured representations. We excluded pure ontology usage and instance-only KG population unless the method also constructed or revised a schema-level grounding artifact. The coded map contains 32 core LLM construction methods, five supporting representation or validator works, and one adjacent enterprise-agent architecture; only core rows support numerical claims.

2 Operational Grounding Framework

Classical ontology engineering defines an ontology as an explicit specification of a shared conceptualization ([Gruber, 1993](#); [Guarino, 1998](#)). That

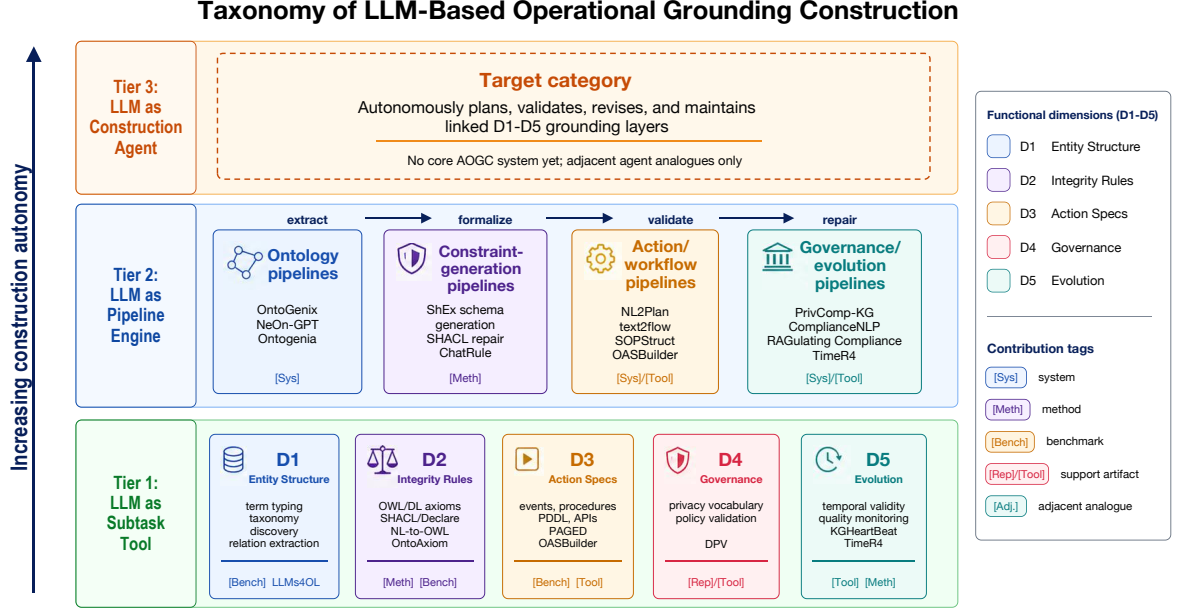


Figure 1: Taxonomy of LLM-based operational grounding construction. Organizes construction methods by increasing autonomy, from subtask tools to fixed pipeline engines and the still-open target of construction agents.

definition remains central, but enterprise agents require a wider object than a conceptual vocabulary. A claims-processing agent, for example, needs to know what a CLAIM is, which fields make it valid, which tools can submit or approve it, which roles may invoke those tools, and which version of each rule applies to the incident under review.

We model the operational grounding layer as $\mathcal{O} = \langle V, C, A, G, M \rangle$. V is the vocabulary layer: shared classes, attributes, relations, identifiers, and mappings to data or API types. C is the constraint and inference layer: axioms, validation shapes, rules, and consistency checks over V . A is the action and process layer: executable operations, typed parameters, preconditions, effects, control flow, and failure modes. G is the governance layer: explicit permissions, prohibitions, obligations, actor roles, audit rules, and regulatory conditions over V , C , and A . M is the maintenance layer: provenance, versions, change operations, deprecations, migrations, and regression tests over time.

The framework is requirements-neutral but not semantics-neutral. The same statement “submitted claims must include documentation” can become an ontology axiom, a validation shape, a JSON Schema field constraint, a SQL NOT NULL check, or a workflow guard. These choices differ under open- versus closed-world assumptions, inference versus validation, and compile-time versus runtime enforcement. Similarly, a planning precondition,

an API parameter schema, and a workflow gateway all carry action knowledge but not the same transition semantics. An AOGC method must therefore specify which part of $\mathcal{O}$ it constructs and how the artifact is validated in use.

The five dimensions are interdependent. Governance rules reference actions performed by roles over entities; action preconditions and effects rely on entity schemas and integrity constraints; maintenance must preserve prior commitments while updating entities, rules, actions, and policies. This interdependence is why we distinguish AOGC from ordinary extraction: the hard problem is not finding candidate terms or relations, but producing linked artifacts that an agent, planner, validator, or policy engine can use directly. The unit of success is therefore the consistency of references across artifacts, not the standalone quality of any one ontology, rule file, process model, or API specification.

The framework also clarifies the NLP challenges in each dimension. D1 requires schema-level information extraction: inducing entity *types* and relations, not only extracting mentions. D2 requires translating ambiguous normative and descriptive language into formal constraints while checking satisfiability and cross-rule interactions. D3 requires procedural understanding, state-transition reasoning, and executable specification of actions. D4 requires regulatory and policy NLU with grounded references to V , C , and A . D5 turns

construction into continual maintenance: detecting changed source evidence, performing impact analysis, and generating targeted updates without breaking existing agent behavior.

Construction setting. The input to AOGC is rarely a single document but a collection of sources with different authority levels and update cycles. A database schema may define claim identifiers and foreign keys; an API reference may expose a `submitClaim` endpoint; a workflow diagram may show that senior review precedes payment above a threshold; a policy manual may state who may approve exceptions; and a release note may change the threshold. No source alone contains the grounding layer. The construction system must identify overlapping concepts, decide which source controls which decision, and preserve evidence so that generated artifacts can be audited.

This setting changes what counts as success. A generated class hierarchy is useful only if downstream actions and policies can refer to it. A generated API schema is incomplete if the agent cannot determine when the call is valid. A generated policy rule is unsafe if it references an action name or role that does not exist in V or A . A generated migration is risky if it updates one rule while leaving stale validation tests behind. AOGC evaluation therefore should combine structural checks with behavioral ones: whether the artifact parses, whether validators or reasoners accept it, whether an agent using it rejects invalid actions, and whether provenance makes disputes inspectable.

Relation to ontology engineering. Much of this agenda is rooted in established ontology-engineering practice: competency questions, modularization, ontology design patterns, reuse, pitfall detection, ontology-mediated access, and evolution methods (Studer et al., 1998). AOGC does not replace these ideas; it changes the construction regime. Instead of asking an expert to hand-author the ontology, the system must infer or revise linked artifacts from language-heavy, multi-source evidence and then demonstrate that an agent can act under them. This makes traditional methodological safeguards more important, not less: competency questions become downstream agent tasks, validation shapes become runtime guards, and versioning becomes part of the agent’s safety case.

3 Taxonomy of Construction Methods

We classify methods by the amount of construction autonomy delegated to the LLM. The levels parallel LLM-system trends from task automation to agentic workflows, but the criterion is specific to grounding-layer construction: who decides which artifact to build, how construction steps are sequenced, and how failures are repaired. Autonomy in this taxonomy is thus not a claim about model intelligence in general; it is a claim about delegated responsibility for construction decisions over the grounding layer. Figure 1 maps the tiers to the five grounding dimensions.

Tier 1: LLM as subtask tool. Tier 1 work supplies the local operations from which AOGC systems are built. For D1, prompted, QA-formulated, retrieval-augmented, instruction-tuned, and code-style extraction methods recover entities, relations, events, triples, and schema-conditioned records (Agrawal et al., 2022; Wei et al., 2023; Zhang et al., 2023; Wan et al., 2023; Wang et al., 2023; Sainz et al., 2024; Bi et al., 2024; Papaluca et al., 2024; van Cauter and Yakovets, 2024; Zhang et al., 2024; Zheng et al., 2024), and LLMs4OL provides controlled ontology-learning subtasks (Babaei Giglou et al., 2023). For D2, LLMs propose ontology axioms (OWL/DL), identify candidate constraints, translate recommendations into validation shapes (SHACL), and extract business-rule templates (Mateiu and Groza, 2023; Bakker et al., 2025; Westermann et al., 2025; Park et al., 2026; Yang et al., 2025). For D3, bounded methods extract events, procedural steps, preconditions and effects, API operations, and tool signatures (Gao et al., 2023; Patil et al., 2024; Lazar et al., 2025; Ni et al., 2025b,a). These methods are essential, but a human or downstream pipeline still decides which candidates enter the grounding layer. In agent-system terms, D3 is the layer that turns extracted knowledge into tool-use and function-calling affordances: not only what tools exist, but which objects they operate on and when they may be invoked.

Tier 2: LLM as pipeline engine. Tier 2 methods sequence several construction stages inside a fixed architecture. In ontology construction, NeOn-GPT, Ontogenia, and OntoGenix combine requirements, concept extraction, hierarchy formation, serialization, syntax checking, reasoner or OOPS! feedback, and expert review (Fathallah et al., 2024; Lippolis et al., 2025; Val-Calvo et al.,

Table 2: Autonomy tiers for LLM-based operational grounding construction.

Tier	LLM role	Human role	Typical output
1: Subtask Tool	Performs one predefined extraction, translation, or validation step	Selects inputs, designs prompts, integrates and accepts output	Terms, relations, triples, candidate axioms, local rules
2: Pipeline Engine	Runs a fixed multi-step construction and checking workflow	Defines domain scope and accepts/rejects final artifact	Ontology drafts; validation shapes (SHACL/ShEx); PDDL action domains; workflows; OpenAPI specs
3: Construction Agent	Plans construction strategy, identifies missing artifacts, validates and revises over time	Sets goals and performs acceptance testing	Evolving linked grounding layers

2024). AutoSchemaKG couples triple extraction with dynamic schema induction at web scale (Bai et al., 2025); AutoGraph-R1 optimizes graph construction with reinforcement-learning rewards from downstream RAG performance (Tsang et al., 2025); and intention-KG construction from behavioral sessions models relational intention structure for platform agents (Bai et al., 2026). Constraint pipelines generate validation schemas (ShEx), repair data-shape rules (SHACL), or mine logical KG rules and expose the output to validators or downstream KG tasks (Zhang et al., 2025; Lin et al., 2025; Luo et al., 2023). Planning, workflow, and process-modeling pipelines translate natural-language descriptions into planning domains (PDDL), workflow models (BPMN/POWL), SOP graphs, or procedural flows, and then verify them with planners, soundness checks, simulation, or conformance metrics (Silver et al., 2024; Guan et al., 2023; Gestrin et al., 2024; Oswald et al., 2024; Smirnov et al., 2024; Kourani et al., 2025; Hörner et al., 2026; Ying et al., 2026; Garg et al., 2025). API-documentation systems such as SpeCrawler, OASBuilder, ToolFactory, and Doc2Agent recover API and tool contracts (OpenAPI) from heterogeneous documentation (Lazar et al., 2024, 2025; Ni et al., 2025b,a). Governance and maintenance pipelines remain thinner, but privacy and compliance KGs and temporal or repair systems already supply partial D4 and D5 components (Garza et al., 2024; Guo et al., 2026; Agarwal et al., 2025; Lin et al., 2025; Qian et al., 2024). Tier 2 is the current frontier because external validators make generation operationally testable rather than merely fluent.

The most important pattern in Tier 2 is the pairing of LLM generation with a non-neural checker. Ontology pipelines call RDF/OWL parsers, reasoners, and pitfall detectors; validation-shape systems execute SHACL or ShEx validators; planning sys-

tems use domain validators or planners; workflow systems use soundness checks, simulation, or conformance tests; API systems use OpenAPI validators and manual evidence alignment. This pattern gives the LLM a repair signal that is more precise than another natural-language critique. It also exposes a boundary: checkers are artifact-specific. A planner can reject malformed preconditions but cannot determine whether a regulatory exception is missing. A SHACL-style validator can flag an invalid graph but cannot infer whether an API operation should be available to a trainee adjuster. AOGC therefore needs multiple validators coordinated through the shared grounding tuple.

Tier 3: LLM as construction agent. Tier 3 requires more than iterative repair. A construction agent must decide which grounding artifacts are missing, choose sources and validators, revise its strategy after failures, and maintain the linked layer. We found no core LLM construction method satisfying this criterion across D1–D5. Adjacent work points toward the target: end-to-end planning agents can adapt their planning abstractions under verifier feedback, and enterprise architectures such as FAOS show how ontology-constrained runtime grounding might be coupled to agents (Luong Tuan and Sanyal, 2026). These systems do not yet autonomously construct and maintain entity, constraint, action, governance, and maintenance artifacts from messy enterprise sources.

Tier 3 requires a different treatment of uncertainty. A subtask extractor can return a candidate list; a pipeline can fail validation and retry; a construction agent must decide when the source evidence is insufficient and when to ask a human for clarification. This is important for D4 and D5, where silently inventing missing permissions or migrations can be worse than leaving the artifact incomplete. In our coding, systems that merely

use agentic prompting or iterative feedback are not Tier 3 unless they choose construction goals and maintenance actions over the grounding layer.

The trend is clear. Early work asked whether LLMs could extract or generate ontology components. Recent work wraps LLM generation with symbolic or executable checkers: RDF/OWL parsers, reasoners, SHACL or ShEx validators, planners, process-model soundness checks, simulation, and OpenAPI validation. This is the right direction for AOGC because the output is not compared with a reference string but parsed, executed, or used in a downstream task. The limitation is integration. D1 and D2 systems omit actions, planning systems assume formalizable domains, workflow systems focus on process structure, and compliance systems output QA or gap reports rather than enforceable governance layers.

4 Gap Analysis

We separate two kinds of gap. *Content gaps* concern which artifacts are not yet constructed. *Realism gaps* concern whether methods that target the right artifacts are evaluated under conditions resembling enterprise deployment.

D1 is broad but shallow. Entity and relationship extraction is the most mature dimension: LLMs can recover terms, taxonomic edges, triples, and schema-conditioned records, and Tier 2 ontology pipelines assemble them into draft ontologies. Recent schema-induction systems such as AutoSchemaKG and relational intention graphs from behavioral sessions expand D1 construction beyond document salience (Bai et al., 2025, 2026). The gap is operational relevance. Enterprise agents need roles, fiscal periods, forms, queues, systems, exception states, audit entities, and data and API mappings that may be only implicit in documents. Current D1 methods extract what is salient, not what an agent must distinguish to act correctly.

D2 is improving but fragmented. Constraint construction now covers ontology axioms (OWL/DL), validation shapes (SHACL/ShEx), business-rule extraction, KG rules, and repair loops (Mateiu and Groza, 2023; Bakker et al., 2025; Fathallah et al., 2024; Zhang et al., 2025; Lin et al., 2025; Luo et al., 2023). The strongest systems validate artifacts with reasoners, SHACL or ShEx validators, SQL compilation, or expert review. Yet most constraints remain local: subclass, domain or

range, disjointness, simple cardinality, templates, or shapes over an existing graph. Few methods construct a satisfiable, regression-tested D2 layer tied to D1, D3, D4, and D5.

D3 is the critical integration gap. Action specification turns a vocabulary into an operational substrate. Planning work generates or repairs PDDL-style action operators; workflow work maps procedures into DAGs and simulation-ready process graphs; API work recovers callable OpenAPI operations and tool contracts (Guan et al., 2023; Gestrin et al., 2024; Oswald et al., 2024; Kourani et al., 2025; Hörner et al., 2026; Ying et al., 2026; Garg et al., 2025; Lazar et al., 2025). The gap lies at their intersection: enterprise action knowledge is distributed across policies, API references, workflow configurations, manuals, tickets, and code, and no coded method fuses these into a single action layer aligned with entities, constraints, governance, and versions. For agent researchers, this is the construction counterpart of tool use: D3 determines whether function-calling choices are grounded in valid enterprise state rather than in a prompt-time list of callable names.

D4 and D5 remain underdeveloped. Governance is mostly represented by compliance QA, privacy vocabularies, and validation subtasks (Garza et al., 2024; Guo et al., 2026; Agarwal et al., 2025; Pandit et al., 2024; Hernandez and Pandit, 2026); a deployable D4 layer must bind permissions, prohibitions, obligations, exceptions, and audit evidence to concrete D1 entities and D3 actions. In deployment terms, D4 is the alignment layer closest to the organization: it determines which otherwise valid actions are permitted, prohibited, auditable, or escalation-worthy for a given actor and context. D5 is thinner still: temporal validity, temporal KGQA, KG repair, ontology versioning, RDF graph deltas, quality monitoring, and continual KG learning provide components (Soulard et al., 2025; Qian et al., 2024; Hemid et al., 2024; Sowinski et al., 2025; Pellegrino et al., 2025; Li et al., 2024), but AOGC requires targeted updates over $\langle V, C, A, G, M \rangle$, not one-shot reconstruction.

The realism gap is as important as the content gap. Academic inputs are often clean, single-source, and internally consistent; enterprise sources are heterogeneous, multi-authored, stale in places, and contradictory. Academic evaluation often reconstructs a known ontology; enterprise deployment needs a fit-for-purpose artifact when no gold

Table 3: Condensed evidence map for the coded corpus. Counts refer to 32 LLM construction methods unless noted. “Any” includes partial or full coverage; “full” means the artifact is a constructed or evaluated target.

Question	Count	Strongest evidence	Remaining limitation
Autonomy	7 T1, 25 T2, 0 T3	Fixed pipelines with reasoners, validators, planners, simulators, or OpenAPI checks	No system chooses missing D1–D5 construction and maintenance actions autonomously
D1 entities	31 any, 6 full	IE, taxonomy, KG, schema-induction, and ontology-generation systems	Operational relevance is usually inferred from text salience, not agent task needs
D2 rules	25 any, 9 full	OWL/DL axioms, SHACL/ShEx shapes, business rules, KG rules	Constraints remain local and weakly connected to actions, policies, and updates
D3 actions	15 any, 9 full	PDDL planning models, workflow/process models, SOP graphs, OpenAPI/tool contracts	Planning, workflow, and API artifacts are rarely aligned into one action layer
D4 governance	6 any, 0 full	Privacy/compliance KGs, SHACL validation, regulatory QA	Policy artifacts are not linked to enforceable action control over D1–D3
D5 maintenance	6 any, 0 full	Temporal validity, KG repair, ontology versioning, RDF deltas	No targeted update loop over the whole grounding layer

Table 4: Realism audit for the 32 core rows. “Any” means the property is evaluated at least once; “full” means it is central to the evaluation.

Evaluation property	Any	Full
Heterogeneous or multi-source input	7	2
No fixed gold ontology/schema reference	12	5
External formal validator or executor	17	12
Downstream task or agent-behavior test	9	5
Contradiction localization or resolution	0	0
Gap detection, abstention, or clarification	0	0
Incremental update after source change	3	0
Production-scale continuous operation	0	0

ontology exists. Benchmarks usually assume all necessary knowledge is present; enterprise practice includes tacit procedures and missing policy details. Finally, evaluation is usually single-shot, while operational grounding layers evolve continuously. The result is an evidential mismatch: strong benchmark performance on taxonomy or axiom reconstruction does not by itself show that a method can support agent behavior in a real organization.

This mismatch is visible even when a paper reports an executable artifact. Planner success shows that a generated PDDL domain can support a class of planning tasks, but not that the operators align with an enterprise API or approval policy. SHACL validation shows that a data graph satisfies a set of shapes, but not that the shapes cover all agent-relevant invariants. OpenAPI validity shows that an operation contract is syntactically usable, but not when an agent should call the operation or whether the call is permitted. These evaluations are necessary building blocks; they become enterprise evidence only when combined into end-to-end tests over realistic sources and agent decisions.

The absence of contradiction and gap handling is significant. In deployments, the correct answer may be that two sources disagree or that no source licenses a required rule. AOGC systems should be evaluated not only on generation accuracy but also

on *non-generation*: abstaining, asking for clarification, isolating conflicting passages, or producing a partial artifact with explicit unresolved assumptions. Current benchmarks rarely reward these behaviors, so systems have little incentive to distinguish between unsupported but plausible completions and evidence-grounded construction.

Four root causes explain the pattern. The relevant research communities are fragmented across ontology engineering, NLP, planning, software engineering, business-process management, and legal and regulatory AI. Clean benchmarks with gold references are easier to publish than messy downstream task evaluations. Enterprise documentation is proprietary, pushing researchers toward public proxies that lack the hardest characteristics. Finally, academic metrics often optimize reference similarity, while enterprise success is measured by whether an agent takes valid, permitted, and auditable actions.

5 Research Agenda

Unified entity–action construction. The highest-impact direction is joint extraction of entities, relations, constraints, actions, and state variables. An extracted action schema should bind parameters to D1 classes or API objects, express preconditions and effects over D1 predicates, and attach D2 invariants that must hold before and after execution. Action construction should expose missing entity distinctions, and constraint violations should trigger action repair. Validation should combine formal checks with behavioral tests: planning success, invalid-action rejection, constrained execution, workflow simulation, and expert review of execution traces.

Governance formalization with provenance. AOGC systems must translate policy and regulatory language into enforceable rules connected to enti-

ties and actions. This requires legal and regulatory NLU for modality and exceptions, multi-document reasoning over cross-references, and grounded generation that does not invent nonexistent ontology elements. Because governance errors are asymmetric, generated rules should include source provenance, confidence, unresolved exceptions, and calibrated abstention when human approval is required.

Enterprise-realistic benchmarks. The field needs public benchmarks that approximate enterprise conditions while remaining shareable. Open-source projects with documentation, APIs, access control, issue histories, and release changes are one plausible substrate. Evaluation should include action-validity tests, permission and obligation tests, contradiction-localization tasks, source-change updates, regression preservation, and human adjudication for missing or tacit knowledge. Metrics should measure invalid-action rejection, valid-action completion, precondition and effect consistency, policy-enforcement accuracy, provenance precision, contradiction localization, targeted-update success, and calibrated clarification behavior.

Such benchmarks should separate three scoring layers. *Artifact fitness* asks whether generated vocabularies, SHACL-style validation rules, PDDL-style action models, process models, API specifications, and policies parse and pass the appropriate validators. *Behavioral fitness* asks whether an agent grounded in those artifacts chooses valid actions, refuses invalid ones, and supplies the right evidence for its decisions. *Maintenance fitness* asks whether the system updates the grounding layer after a source change without breaking previously valid behavior. A method that performs well on artifact fitness but fails on behavioral fitness is a structured-generation method, not yet an operational grounding method.

Incremental maintenance. Operational grounding must become a living artifact. Methods should detect changed source evidence, identify the affected entities, actions, constraints, and policies, regenerate only the affected pieces, and preserve unaffected behavior through regression tests. This direction connects ontology evolution, temporal KGs, software migration, and agent evaluation, but requires benchmarks that release source changes over time rather than a single static corpus.

Conflict-aware multi-source integration. Practical systems must ingest prose, tables, diagrams, schemas, API specifications, workflow definitions, code comments, tickets, and human clarification. They must align overlapping evidence, retain provenance, and explicitly surface conflicts rather than silently averaging them away. For AOGC, uncertainty is a first-class output: a system that flags contradictory approval thresholds and asks for resolution is more deployable than one that emits a confident but unsupported rule.

Deployment-time evaluation. If AOGC is meant to automate the work now performed by forward-deployed engineers and enterprise platform teams, evaluation should include deployment metrics as well as artifact metrics. Useful measures include time to first safe agent action in a new domain, engineering hours per customer or workflow, regression rate after a source-policy change, unresolved-ambiguity rate, invalid-action incident rate, and the fraction of agent failures attributable to missing or stale grounding. These metrics tie construction quality to the business and safety bottleneck that motivates enterprise agent deployment.

A reference architecture. A practical near-term AOGC system would likely be hybrid. Retrieval and document parsing would collect evidence spans and structured source artifacts. Schema induction would propose D1 entities and relations. Constraint and action generators would draft SHACL-style validation rules and PDDL-style action models against that schema. Policy formalizers would bind permissions and obligations to the same identifiers. Validators, planners, simulators, and policy engines would provide typed error reports; recent context-engineering work on executable instruction constraints is a useful adjacent signal for this validator-backed design (Sharma, 2026). A provenance layer would preserve source links and confidence, while a maintenance controller would compare new source versions against the accepted grounding layer. The LLM’s role would not be a single prompt call but an orchestrated constructor operating under symbolic and behavioral feedback.

6 Conclusion

LLMs can already construct useful grounding fragments: entity structures, validation rules, planning or workflow artifacts, and API or tool specifications. AOGC asks whether those fragments can

be linked, validated, governed, and maintained as $\mathcal{O} = \langle V, C, A, G, M \rangle$: vocabulary, constraints, actions, governance, and maintenance for enterprise agents. The coded literature remains concentrated in Tier 1 subtasks and Tier 2 fixed pipelines, with D1 strongest, D4 and D5 weakest, and little evaluation on heterogeneous sources, contradictions, tacit gaps, incremental change, or downstream agent behavior. The agenda is therefore integrative: entity–action co-construction, governance formalization, realistic benchmarks, incremental maintenance, and conflict-aware synthesis.

Limitations

This survey is a structured scoping review of LLM-based work published or publicly available through 21 May 2026. As with any fast-moving area, some industry systems and recent deployments may not be fully visible in the public literature, and tier or dimension labels involve judgment calls when papers construct one artifact while using another for validation or evaluation. We therefore use the reported counts as sample-level evidence about the coded literature, and read the gap analysis as a research map for evaluating operational grounding across domains with different regulatory exposure, automation tolerance, data availability, and review workflows.

References

- Bhavik Agarwal, Hemant Sunil Jomraj, Simone Kaplunov, Jack Krolick, and Viktoria Rojkova. 2025. Regulating compliance: A multi-agent knowledge graph for regulatory qa. In *arXiv preprint*.
- Monica Agrawal, Stefan Hegselmann, Hunter Lang, Yoon Kim, and David Sontag. 2022. [Large language models are few-shot clinical information extractors](#). In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 1998–2022, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.
- Anthropic. 2024. Model context protocol specification. <https://spec.modelcontextprotocol.io/specification/2024-11-05/>. Protocol revision 2024-11-05.
- Anthropic. 2026. Building a new enterprise AI services company with blackstone, hellman & friedman, and goldman sachs. <https://www.anthropic.com/news/enterprise-ai-services-company>. Accessed 22 May 2026.
- Hamed Babaei Giglou, Jennifer D’Souza, and Sören Auer. 2023. LLMs4OL: Large language models for ontology learning. In *International Semantic Web Conference*, pages 408–427. Springer.
- Jiaxin Bai, Wei Fan, Qi Hu, Qing Zong, Chunyang Li, Hong Ting Tsang, Hongyu Luo, Yauwai Yim, Haoyu Huang, Xiao Zhou, Feng Qin, Tianshi Zheng, Xi Peng, Xin Yao, Huiwen Yang, Leijie Wu, Yi Ji, Gong Zhang, Renhai Chen, and Yangqiu Song. 2025. [AutoSchemaKG: Autonomous knowledge graph construction through dynamic schema induction from web-scale corpora](#). arXiv preprint arXiv:2505.23628.
- Jiaxin Bai, Zhaobo Wang, Junfei Cheng, Dan Yu, Zerui Huang, Weiqi Wang, Xin Liu, Chen Luo, Yanming Zhu, Bo Li, and Yangqiu Song. 2026. [Intention knowledge graph construction for user intention relation modeling](#). In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 466–484.
- Roos M. Bakker, Daan L. Di Scala, Maaike H. T. de Boer, and Stephan A. Raaijmakers. 2025. [Ontology learning with LLMs: A benchmark study on axiom identification](#). *CoRR*, abs/2512.05594.
- Zhen Bi, Jing Chen, Yinuo Jiang, Feiyu Xiong, Wei Guo, Huajun Chen, and Ningyu Zhang. 2024. [CodeKGC: Code language model for generative knowledge graph construction](#). *ACM Transactions on Asian and Low-Resource Language Information Processing*, 23(3):1–16.
- Haonan Bian. 2025. [Llm-empowered knowledge graph construction: A survey](#). *Preprint*, arXiv:2510.20345.
- Valentina Anita Carriero, Antonia Azzini, Ilaria Baroni, Mario Scrocca, and Irene Celino. 2024. [Human evaluation of procedural knowledge graph extraction from text with large language models](#). *CoRR*, abs/2412.03589.
- Nathanael Chambers. 2013. [Event schema induction with a probabilistic entity-driven model](#). In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1797–1807, Seattle, Washington, USA. Association for Computational Linguistics.
- Nathanael Chambers and Dan Jurafsky. 2008. [Unsupervised learning of narrative event chains](#). In *Proceedings of ACL-08: HLT*, pages 789–797, Columbus, Ohio. Association for Computational Linguistics.
- Qizhou Chen, Dakan Wang, Taolin Zhang, Zaoming Yan, Chengsong You, Chengyu Wang, and Xiaofeng He. 2025. Uniedit: A unified knowledge editing benchmark for large language models. In *arXiv preprint*.
- Jackie Chi Kit Cheung, Hoifung Poon, and Lucy Vanderwende. 2013. [Probabilistic frame induction](#). In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 837–846, Atlanta, Georgia. Association for Computational Linguistics.

- Yizhe Chi, Deyao Hong, Dapeng Jiang, Tianwei Luo, Qinhui Na, Kaisen Yang, and Boshi Zhang. 2026. [Frontier-Eng: Benchmarking self-evolving agents on real-world engineering tasks with generative optimization](#). *arXiv preprint*.
- Weihong Du, Wenrui Liao, Hongru Liang, and Wenqiang Lei. 2024. [PAGED: A benchmark for procedural graphs extraction from documents](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 10829–10846.
- Kutluhan Erol, James Hendler, and Dana S. Nau. 1994. [UMCP: A sound and complete procedure for hierarchical task-network planning](#). In *Proceedings of the Second International Conference on Artificial Intelligence Planning Systems*, pages 249–254.
- Nadeen Fathallah, Arunav Das, Stefano De Giorgis, Andrea Poltronieri, Peter Haase, and Liubov Kovriguina. 2024. Neon-gpt: a large language model-powered pipeline for ontology learning. In *European Semantic Web Conference*, pages 36–50. Springer.
- Jun Gao, Huan Zhao, Changlong Yu, and Ruifeng Xu. 2023. [Exploring the feasibility of ChatGPT for event extraction](#). *CoRR*, abs/2303.03836.
- Deepeka Garg, Sihan Zeng, Sumitra Ganesh, and Leo Ardon. 2025. [Generating structured plan representation of procedures with LLMs](#). *CoRR*, abs/2504.00029.
- Daniel Garijo, María Poveda-Villalón, Elvira Amador-Domínguez, Ziyuan Wang, Raúl García-Castro, and Oscar Corcho. 2024. LLMs for ontology engineering: A landscape of tasks and benchmarking challenges. In *HGAIS@ ISWC*.
- Leon Garza, Lavanya Elluri, Anantaa Kotal, Aritran Piplai, Deepti Gupta, and Anupam Joshi. 2024. Privcomp-kg : Leveraging knowledge graph and large language models for privacy policy compliance verification. In *arXiv preprint*.
- Sandra Geisler, CINZIA CAPPIELLO, Irene Celino, David Chaves-Fraga, Anastasia Dimou, Ana Iglesias-Molina, Maurizio Lenzerini, Anisa Rula, Dylan Van Assche, Sascha Welten, and Maria-Esther Vidal. 2025. From genesis to maturity: Managing knowledge graph ecosystems through life cycles. In *Zenodo*.
- Elliot Gestrin, Marco Kuhlmann, and Jendrik Seipp. 2024. [NL2Plan: Robust LLM-driven planning from minimal text descriptions](#). *CoRR*, abs/2405.04215.
- Michael Grohs, Luka Abb, Nourhan Elsayed, and Jana-Rebecca Rehse. 2024. [Large language models can accomplish business process management tasks](#). In *Business Process Management Workshops*, pages 453–465. Springer.
- Thomas R. Gruber. 1993. [A translation approach to portable ontology specifications](#). *Knowledge Acquisition*, 5(2):199–220.
- Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. 2023. [Leveraging pre-trained large language models to construct and utilize world models for model-based task planning](#). *CoRR*, abs/2305.14909.
- Nicola Guarino. 1998. Formal ontology and information systems. In *Proceedings of the First International Conference on Formal Ontology in Information Systems*, pages 3–15. IOS Press.
- Honghao Gui, Lin Yuan, Hongbin Ye, Ningyu Zhang, Mengshu Sun, Lei Liang, and Huajun Chen. 2024. [IEPile: Unearthing large scale schema-conditioned information extraction corpus](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 127–146, Bangkok, Thailand. Association for Computational Linguistics.
- Dongxin Guo, Jikun Wu, and S. Yiu. 2026. Complain-cenlp: Knowledge-graph-augmented rag for multi-framework regulatory gap detection. In *Proceedings of the Association for Computational Linguistics*.
- Ahmad Hemid, Waleed Shabbir, Abderrahmane Khiat, Christoph Lange, Christoph Quix, and Stefan Decker. 2024. Ontoeditor: Real-time collaboration via distributed version control for ontology development. In *Lecture Notes in Computer Science*.
- J Hernandez and HJ Pandit. 2026. Explainable validation of data sharing agreements using dpv, shacl, and human-in-the-loop review. In *Proceedings of the Extended Semantic Web Conference*.
- Luca Franziska Hörner, Maximilian Möller, and Manfred Reichert. 2026. [Automatically generating BPMN 2.0 process models from natural language process descriptions: Challenges, framework, quality assessment](#). *Business & Information Systems Engineering*, 68(1):59–83.
- Ian Horrocks, Peter F. Patel-Schneider, Harold Boley, Said Tabet, Benjamin Grosz, and Mike Dean. 2004. [SWRL: A semantic web rule language combining OWL and RuleML](#). W3C Member Submission.
- Yongli Hu, Alan Liew, Linhao Luo, Shirui Pan, Kai Sun, Jiapu Wang, Wei Wei, and Baocai Yin. 2024. Large language models-guided dynamic adaptation for temporal knowledge graph reasoning. In *Advances in Neural Information Processing Systems* 37.
- Maxime Jakubowski and Jan Van den Bussche. 2024. Compiling SHACL into SQL. In *Proceedings of the International Semantic Web Conference*.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. [SWE-bench: Can language models resolve real-world github issues?](#) In *The Twelfth International Conference on Learning Representations*.

- Holger Knublauch and Dimitris Kontokostas. 2017. [Shapes constraint language \(SHACL\)](#). W3C Recommendation.
- Humam Kourani, Daniel Schuster, and Wil M. P. van der Aalst. 2024. [Process modeling with large language models](#). *CoRR*, abs/2403.07541.
- Humam Kourani, Daniel Schuster, and Wil M. P. van der Aalst. 2025. [Evaluating large language models on business process modeling: Framework, benchmark, and self-improvement analysis](#). *Software and Systems Modeling*.
- Emanuele La Malfa, Ping Zhu, Samuele Marro, Sara Bernardini, and Michael Wooldridge. 2025. [End-to-end PDDL planning with hardcoded and dynamic agents](#). *CoRR*, abs/2512.09629.
- Koren Lazar, Matan Vetzler, Kiran Kate, Jason Tsay, David Boaz, Himanshu Gupta, Avraham Shinnar, Rohith D. Vallam, David Amid, Esther Goldbraich, Jim Laredo, and Ateret Anaby Tavor. 2025. [Generating OpenAPI specifications from online API documentation with large language models](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics: Industry Track*.
- Koren Lazar, Matan Vetzler, Guy Uziel, David Boaz, Esther Goldbraich, David Amid, and Ateret Anaby-Tavor. 2024. [SpeCrawler: Generating OpenAPI specifications from API documentation using large language models](#). *CoRR*, abs/2402.11625.
- Bo Li, Gexiang Fang, Yang Yang, Quansen Wang, Wei Ye, Wen Zhao, and Shikun Zhang. 2023a. [Evaluating ChatGPT’s information extraction capabilities: An assessment of performance, explainability, calibration, and faithfulness](#). *Preprint*, arXiv:2304.11633.
- Guozheng Li, Peng Wang, and Wenjun Ke. 2023b. [Re-visiting large language models as zero-shot relation extractors](#). In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 6877–6892, Singapore. Association for Computational Linguistics.
- Peng Li, Tianxiang Sun, Qiong Tang, Hang Yan, Yuanbin Wu, Xuanjing Huang, and Xipeng Qiu. 2023c. [CodeIE: Large code generation models are better few-shot information extractors](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15339–15353, Toronto, Canada. Association for Computational Linguistics.
- Xiaoxuan Li, Lin Ni, Xin Wang, Yitong Tang, Ruoxuan Li, Jiamou Liu, and Zhongsheng Wang. 2025a. [LLM-based business process models generation from textual descriptions](#). In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 523–533.
- Yifei Li, Lingling Zhang, Hang Yan, Tianzhe Zhao, Zihan Ma, Muye Huang, and Jun Liu. 2025b. [Sage: Scale-aware gradual evolution for continual knowledge graph embedding](#). In *arXiv preprint*.
- Zhuofeng Li, Haoxiang Zhang, Qiannan Zhang, Ziyi Kou, and Shichao Pei. 2024. [Learning from novel knowledge: Continual few-shot knowledge graph completion](#). In *Proceedings of the ACM International Conference on Information and Knowledge Management*.
- Tung-Wei Lin, Gabe Fierro, Han Li, Tianzhen Hong, Pierluigi Nuzzo, and Alberto Sangiovanni-Vincentelli. 2025. [Systematic evaluation of knowledge graph repair with large language models](#). *CoRR*, abs/2507.22419.
- Anna Sofia Lippolis, Mohammad Javad Saeedizade, Robin Keskisärkkä, Sara Zuppiroli, Miguel Ceriani, Aldo Gangemi, Eva Blomqvist, and Andrea Giovanni Nuzzolese. 2025. [Ontology generation using large language models](#). In *European Semantic Web Conference*, pages 321–341. Springer.
- Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. 2023a. [LLM+P: Empowering large language models with optimal planning proficiency](#). *CoRR*, abs/2304.11477.
- Siwei Liu, Jinyuan Fang, Han Zhou, Yingxu Wang, and Zaiqiao Meng. 2025. [SEW: Self-evolving agentic workflows for automated code generation](#). *CoRR*, abs/2505.18646.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2023b. [AgentBench: Evaluating llms as agents](#). *arXiv preprint arXiv:2308.03688*.
- Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. 2026. [SkillForge: Forging domain-specific, self-evolving agent skills in cloud technical support](#). *CoRR*, abs/2604.08618.
- Linhao Luo, Jiaxin Ju, Bo Xiong, Yuan-Fang Li, Ghohamreza Haffari, and Shirui Pan. 2023. [ChatRule: Mining logical rules with large language models for knowledge graph reasoning](#). *CoRR*, abs/2309.01538.
- Thanh Luong Tuan and Abhijit Sanyal. 2026. [Ontology-constrained neural reasoning in enterprise agentic systems: A neurosymbolic architecture for domain-grounded AI agents](#). *arXiv preprint*.
- Yubo Ma, Yixin Cao, Yong Hong, and Aixun Sun. 2023. [Large language model is not a good few-shot information extractor, but a good reranker for hard samples!](#) In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10572–10601, Singapore. Association for Computational Linguistics.

- David Martin, Mark Burstein, Jerry Hobbs, Ora Lassila, Drew McDermott, Sheila McIlraith, Srinu Narayanan, Massimo Paolucci, Bijan Parsia, Terry Payne, Evren Sirin, Naveen Srinivasan, and Katia Sycara. 2004. [OWL-S: Semantic markup for web services](#). W3C Member Submission.
- Patricia Mateiu and Adrian Groza. 2023. [Ontology engineering with large language models](#). *CoRR*, abs/2307.16699.
- Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. 1998. [PDDL: The planning domain definition language](#). Technical report, Yale Center for Computational Vision and Control.
- McKinsey Global Institute. 2023. [The economic potential of generative AI: The next productivity frontier](#). Technical report, McKinsey & Company.
- Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. [GAIA: A benchmark for general ai assistants](#). In *The Twelfth International Conference on Learning Representations*.
- Xinyi Ni, Haonan Jian, Qiuyang Wang, Vedanishi Chetan Shah, and Pengyu Hong. 2025a. [Doc2Agent: Scalable generation of tool-using agents from API documentation](#). *CoRR*, abs/2506.19998.
- Xinyi Ni, Qiuyang Wang, Yukun Zhang, and Pengyu Hong. 2025b. [ToolFactory: Automating tool generation by leveraging LLM to understand REST API documentations](#). *CoRR*, abs/2501.16945.
- Andrea Giovanni Nuzzolese, Francesco Poggi, Luca Bassi, and Monica Palmirani. 2025. Semantic technologies for global governance: A hybrid ai approach to tracking and monitoring who resolutions. In *Lecture Notes in Computer Science*.
- Object Management Group. 2014. [Business process model and notation \(BPMN\), version 2.0.2](#).
- OpenAI. 2026. Forward deployed engineering at OpenAI. <https://openai.com/business/the-openai-deployment-company/>. Accessed 22 May 2026.
- OpenAPI Initiative. 2024. [OpenAPI specification, version 3.1.1](#).
- James Oswald, Kavitha Srinivas, Harsha Kokel, Junkyu Lee, Michael Katz, and Shirin Sohrabi. 2024. [Large language models as planning domain generators](#). In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, pages 423–431.
- Palantir Technologies. 2021. Form 10-k annual report. <https://www.sec.gov/Archives/edgar/data/1321655/000119312521060650/d65934d10k.htm>. Filed with the U.S. Securities and Exchange Commission.
- HJ Pandit, B Esteves, GP Krog, and P Ryan. 2024. Data privacy vocabulary (dpv)–version 2. In *arXiv preprint*.
- Andrea Papaluca, Daniel Krefl, Sergio Rodriguez Mendez, Artem Lensky, and Hanna Suominen. 2024. [Zero- and few-shots knowledge graph triplet extraction with large language models](#). In *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models (KaLLM 2024)*, pages 12–23, Bangkok, Thailand. Association for Computational Linguistics.
- Gyunam Park, Julian Kofferath, and Minsu Cho. 2026. [Extracting declarative constraints for process modeling from natural language descriptions with large language models](#). *Decision Support Systems*, 203:114627.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. [Gorilla: Large language model connected with massive APIs](#). In *Advances in Neural Information Processing Systems*, volume 37.
- Maria Angela Pellegrino, Anisa Rula, and Gabriele Tuozzo. 2025. Kgheartbeat: A knowledge graph quality assessment tool. In *ESWC Satellite Events*.
- Xinying Qian, Ying Zhang, Yu Zhao, Baohang Zhou, Xuhui Sui, Li Zhang, and Kehui Song. 2024. [Timer⁴: Time-aware retrieval-augmented large language models for temporal knowledge graph question answering](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.
- Oscar Sainz, Iker Garcia-Ferrero, Rodrigo Agerri, Oier Lacalle, German Rigau, and Eneko Agirre. 2024. [GoLLIE: Annotation guidelines improve zero-shot information-extraction](#). In *The Twelfth International Conference on Learning Representations*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. [Toolformer: Language models can teach themselves to use tools](#). In *Advances in Neural Information Processing Systems*, volume 36.
- ServiceNow and Accenture. 2026. ServiceNow and Accenture launch forward deployed engineering program to scale agentic AI across the enterprise. [Press release](#). Accessed 22 May 2026.
- Reshabh K. Sharma. 2026. [ContextCov: Deriving and enforcing executable constraints from agent instruction files](#). *CoRR*, abs/2603.00822.
- Yucheng Shi, Qiaoyu Tan, Xuansheng Wu, Shaochen Zhong, Kaixiong Zhou, and Ninghao Liu. 2024. Retrieval-enhanced knowledge editing in language models for multi-hop question answering. In *Proceedings of the ACM International Conference on Information and Knowledge Management*.

- Tom Silver, Soham Dan, Kavitha Srinivas, Joshua B. Tenenbaum, Leslie Pack Kaelbling, and Michael Katz. 2024. Generalized planning in pddl domains with pretrained large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Pavel Smirnov, Frank Joublin, Antonello Ceravola, and Michael Gienger. 2024. [Generating consistent PDDL domains with large language models](#). *CoRR*, abs/2404.07751.
- Thibaut Soulard, Fatiha Saïs, and Joe Raad. 2025. When facts expire: Benchmarking temporal validity in knowledge graphs. In *Proceedings of the ACM International Conference on Information and Knowledge Management*.
- Piotr Sowinski, Kacper Grzymkowski, and Anastasiya Danilenka. 2025. Jelly-patch: a fast format for recording changes in rdf datasets. In *ISWC Industry, Doctoral Consortium, Posters and Demos*.
- Rudi Studer, V Richard Benjamins, and Dieter Fensel. 1998. Knowledge engineering: Principles and methods. *Data & knowledge engineering*, 25(1-2):161–197.
- Hong Ting Tsang, Jiaxin Bai, Haoyu Huang, Qiao Xiao, Tianshi Zheng, Baixuan Xu, Shujie Liu, and Yangqiu Song. 2025. [AutoGraph-R1: End-to-end reinforcement learning for knowledge graph construction](#). arXiv preprint arXiv:2510.15339.
- Gabriele Tuoizzo. 2025. Quality without borders: A modular approach to unified knowledge graph assessment. In *Proceedings of the International Semantic Web Conference*.
- Mikel Val-Calvo, Mikel Egaña Aranguren, Juan M. Martínez-Hernández, Ginés Almagro-Hernández, Prashant Deshmukh, José Antonio Bernabé-Díaz, Paola Espinoza-Arias, José Luis Sánchez-Fernández, Juergen Mueller, and Jesualdo Tomás Fernández-Breis. 2024. Ontogenix: Leveraging large language models for enhanced ontology engineering from datasets. *Inf. Process. Manag.*
- Zeno van Cauter and Nikolay Yakovets. 2024. [Ontology-guided knowledge graph construction from maintenance short texts](#). In *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models (KaLLM 2024)*, pages 75–84, Bangkok, Thailand. Association for Computational Linguistics.
- Luis Miguel Vieira da Silva, Aljosha Köcher, Felix Gehlhoff, and Alexander Fay. 2024. [On the use of large language models to generate capability ontologies](#). *CoRR*, abs/2404.17524.
- W3C OWL Working Group. 2012. [OWL 2 web ontology language structural specification and functional-style syntax](#). W3C Recommendation.
- Zhen Wan, Fei Cheng, Zhuoyuan Mao, Qianying Liu, Haiyue Song, Jiwei Li, and Sadao Kurohashi. 2023. [GPT-RE: In-context learning for relation extraction using large language models](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3534–3547, Singapore. Association for Computational Linguistics.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. [Voyager: An open-ended embodied agent with large language models](#). *Transactions on Machine Learning Research*.
- Xiao Wang, Weikang Zhou, Can Zu, Han Xia, Tianze Chen, Yuansen Zhang, Rui Zheng, Junjie Ye, Qi Zhang, Tao Gui, Jihua Kang, Jingsheng Yang, Siyuan Li, and Chunsai Du. 2023. [InstructUIE: Multi-task instruction tuning for unified information extraction](#). *Preprint*, arXiv:2304.08085.
- Xiang Wei, Xingyu Cui, Ning Cheng, Xiaobin Wang, Xin Zhang, Shen Huang, Pengjun Xie, Jinan Xu, Yufeng Chen, Meishan Zhang, Yong Jiang, and Wenjuan Han. 2023. [ChatIE: Zero-shot information extraction via chatting with ChatGPT](#). *Preprint*, arXiv:2302.10205.
- Tom Westermann, Aljosha Kocher, and Felix Gehlhoff. 2025. [Automated validation of textual constraints against AutomationML via LLMs and SHACL](#). In *Proceedings of the 30th IEEE International Conference on Emerging Technologies and Factory Automation*.
- Austin Wright, Henry Andrews, Ben Hutton, and Greg Dennis. 2022. [JSON schema validation: A vocabulary for structural validation of JSON](#). Draft 2020-12.
- Yuwei Xia, Ding Wang, Qiang Liu, Liang Wang, Shu Wu, and Xiaoyu Zhang. 2024. Chain-of-history reasoning for temporal knowledge graph forecasting. In *Findings of the Association for Computational Linguistics: ACL 2024*.
- Kaige Xie, Ian Yang, John Gunerli, and Mark Riedl. 2025. [Making large language models into world models with precondition and effect knowledge](#). In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 7532–7545, Abu Dhabi, UAE. Association for Computational Linguistics.
- Chen Yang, Ruping Xu, Ruizhe Li, Bin Cao, and Jing Fan. 2025. [Business as usual: A benchmark and framework for business rule flow modeling with LLMs](#). *CoRR*, abs/2505.18542.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. [\$\tau\$ -bench: A benchmark for tool-agent-user interaction in real-world domains](#). *Preprint*, arXiv:2406.12045.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [ReAct: Synergizing reasoning and acting in language](#)

- models. In *International Conference on Learning Representations*.
- Wangyang Ying, Yanchi Liu, Xujiang Zhao, Wei Cheng, Zhengzhang Chen, Wenchao Yu, Yanjie Fu, and Haifeng Chen. 2026. [Multi-agent procedural graph extraction with structural and logical refinement](#). In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 3021–3034.
- Bohui Zhang, Yuan He, Lydia Pintscher, Albert Meroño-Peñuela, and Elena Simperl. 2025. [Schema generation for large knowledge graphs using large language models](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*.
- Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu. 2026. [CoEvoSkills: Self-evolving agent skills via co-evolutionary verification](#). *CoRR*, abs/2604.01687.
- Kai Zhang, Bernal Jimenez Gutierrez, and Yu Su. 2023. [Aligning instruction tasks unlocks large language models as zero-shot relation extractors](#). In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 794–812, Toronto, Canada. Association for Computational Linguistics.
- Yujia Zhang, Tyler Sadler, Mohammad Reza Taesiri, Wenjie Xu, and Marek Reformat. 2024. [Fine-tuning language models for triple extraction with data augmentation](#). In *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models (KaLLM 2024)*, pages 116–124, Bangkok, Thailand. Association for Computational Linguistics.
- Tianshi Zheng, Zheyang Deng, Hong Ting Tsang, Weiqi Wang, Jiaxin Bai, Zihao Wang, and Yangqiu Song. 2025. [From automation to autonomy: A survey on large language models in scientific discovery](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 17733–17750, Suzhou, China. Association for Computational Linguistics.
- Yi-Kai Zheng, Bi Zeng, Yi-Chun Feng, Lu Zhou, and Yi-Xue Li. 2024. [PLRTE: Progressive learning for biomedical relation triplet extraction using large language models](#). *Journal of Biomedical Informatics*, 159:104738.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and 1 others. 2024a. [WebArena: A realistic web environment for building autonomous agents](#). In *The Twelfth International Conference on Learning Representations*.
- Wenxuan Zhou, Sheng Zhang, Yu Gu, Muhao Chen, and Hoifung Poon. 2024b. [UniversalNER: Targeted distillation from large language models for open named entity recognition](#). In *The Twelfth International Conference on Learning Representations*.
- Xiaochi Zhou, Patrick Bulter, Changxuan Yang, Simon D. Rihm, Thitikarn Angkanaporn, Jethro Akroyd, Sebastian Mosbach, and Markus Kraft. 2026. [Ontology-to-tools compilation for executable semantic constraint enforcement in LLM agents](#). *CoRR*, abs/2602.03439.

A Supplementary Scope and Positioning

This appendix expands the material that the main text compresses for length. The main text preserves the argument and headline counts, while the tables and discussion below provide the detailed survey comparison, corpus accounting, worked insurance example, contribution-type definitions, full coded evidence map, and expanded corpus analysis.

B Risks

AOGC can reduce manual grounding work, but it also introduces deployment risks if generated artifacts are accepted without review. Incorrect schemas can hide required state, incomplete constraints can permit invalid records, and mismatched action or policy models can authorize actions that users or organizations did not intend. Maintenance errors are especially risky because a stale artifact may remain syntactically valid while no longer reflecting a changed API, workflow, or regulation.

These risks motivate the governance requirements emphasized in the survey: source-grounded provenance, explicit uncertainty, validation against symbolic engines, downstream behavior tests, human acceptance gates for high-impact changes, and regression checks after source updates. AOGC systems should be deployed as auditable construction aids before being trusted as autonomous grounding maintainers.

Agent benchmarks such as SWE-bench, WebArena, GAIA, AgentBench, and τ -bench make agent behavior measurable in interactive or tool-using settings (Jimenez et al., 2024; Zhou et al., 2024a; Mialon et al., 2024; Liu et al., 2023b; Yao et al., 2024). They differ from AOGC because the grounding environment is largely supplied by the benchmark designer: repositories, websites, tools, user goals, policies, databases, and scoring functions are already specified. Our survey instead asks how those operational artifacts can themselves be constructed, validated, governed, and maintained from enterprise evidence.

Table 5: Reader’s guide to recurring formalisms. The main text groups these terms by the role they play in operational grounding; this table expands the acronyms for readers outside knowledge representation and software systems.

Term	Role in this survey	Plain-language reading
OWL/RDFS	Ontology and vocabulary languages	Define classes, relations, and type hierarchies for domain objects.
SHACL/ShEx	Data-shape and validation languages	State what a valid graph or record must contain, such as required fields.
JSON Schema	API/data validation language	Specifies required fields and value types for JSON data.
PDDL/HTN	Planning and action-model languages	Describe actions, parameters, preconditions, effects, and task decompositions.
BPMN/POWL/Petri nets	Workflow and process models	Represent ordered business steps, branches, concurrency, and process constraints.
OpenAPI	API specification language	Describes callable endpoints, parameters, request/response schemas, and errors.
RBAC/ABAC/ODRL/Rego	Access-control and policy languages	Encode who may do what, under which roles, attributes, or obligations.
RDF deltas/versioning	Maintenance representation	Record changes to graph-structured grounding artifacts over time.

Table 6: Comparison with nearby surveys and benchmarks. Entries indicate the primary focus of each work rather than whether the topic is mentioned at all.

Work	Agent	Act.	Policy	Realism	Auto.
Garijo et al.	Partial	Peripheral	Peripheral	Limited	No
Bian et al.	Partial	Limited	Limited	Limited	No
LLMs4OL	Subtask	No	No	Benchmark	No
Agent benchmarks	Primary	Primary	Varies	Benchmark	No
Ours	Primary	Primary	Primary	Primary	Primary

C Supplementary Taxonomy Details

Contribution types. Orthogonal to the autonomy tiers, we classify each work by the nature of its primary contribution. **System contributions** provide end-to-end architectures or platforms: complete pipelines from input to structured output. They demonstrate feasibility and integration but may involve limited novelty in individual components. **Method contributions** introduce novel techniques for specific construction challenges, such as prompting strategies, reasoning approaches, validation mechanisms, or optimization methods. They advance particular subtasks but may not demonstrate end-to-end viability. **Benchmark contributions** develop evaluation resources, datasets, metrics, shared tasks, or protocols. They enable reproducible comparison but do not themselves construct ontologies. **Representation contributions** propose new formalisms, intermediate representations, or ontology design patterns optimized for LLM generation or consumption.

Table 7: Corpus accounting for the coded evidence map. Counts are retained records after duplicate removal, not raw search-engine hit counts.

Stage	Records	Kept
Bibliography records retained from search, venue scan, and chaining	113	96 cited
Background standards, prior surveys, and KR/NLP foundations	39	39 background
Candidate construction/evaluation papers screened for D1–D5 relevance	60	38 coded
Excluded from coded map: instance-only KG population or ontology usage	8	–
Excluded from coded map: no LLM construction/validation stage	7	–
Excluded from coded map: duplicate, position-only, or insufficient artifact detail	4	–
Coded evidence map: core/supporting/adjacent rows	38	32/5/1

Integrity-rule construction. The D2 corpus is anchored in textual rules, schema artifacts, and validation resources rather than in entity mentions alone. Inputs include natural-language domain descriptions, competency questions, tabular datasets, knowledge graphs, privacy policies, regulatory requirements, SHACL shapes, and quality reports. OntoGenix uses ontology-generation inputs that contain enough structure to support partial axiom generation (Val-Calvo et al., 2024); SHACL-to-SQL starts from existing SHACL constraints and targets executable database validation (Jakubowski and Van den Bussche, 2024); DPV and DPV-based validation work provide privacy vocabularies and policy-oriented constraint targets (Pandit et al., 2024; Hernandez and Pandit, 2026); and KGHeartBeat-style tools provide diagnostic signals for inconsistency or degradation in constructed

Table 8: Worked mapping of the insurance example into $\mathcal{O} = \langle V, C, A, G, M \rangle$. The same source sentence may produce different artifacts depending on whether the target semantics are inference, validation, action execution, policy enforcement, or maintenance.

Layer	Example evidence	Possible artifact	Semantic caveat
V Vocabulary	Claims involve claimant, policy, incident, adjuster	Vocabulary classes/relations; DB/API type mappings	Open-world conceptual commitments differ from closed-world DB schemas
C Constraints	Submitted claims must include documentation	Required-field validation rule; ontology cardinality; SQL constraint	Validation, inference, and database enforcement are not equivalent
A Actions	Submit, review, approve, escalate, pay claim	API operation; planning action; workflow task/gateway	API signatures do not by themselves specify preconditions/effects
G Governance	Trainees may not approve payouts; high-value claims need senior review	Access-control or policy rule linked to action and claim value	Normative permission/prohibition differs from data-shape validity
M Maintenance	Regulatory update raises the documentation threshold	Versioned migration, provenance record, regression test over affected rules	Updates must preserve audit trails and avoid breaking old workflows

KGs (Pellegrino et al., 2025). More focused D2 work uses bounded natural-language statements for OWL/DL axiom translation (Mateiu and Groza, 2023), existing ontologies for axiom identification (Bakker et al., 2025), and operational documents such as AutomationML recommendations, process descriptions, and business-rule corpora for SHACL, Declare, or condition–action rule extraction (Westermann et al., 2025; Park et al., 2026; Yang et al., 2025). Tier 2 methods add KG evidence for ShEx schema generation, SHACL violation reports for repair, and KG paths for logical rule mining (Zhang et al., 2025; Lin et al., 2025; Luo et al., 2023).

The corresponding outputs are machine-checkable constraints or rule-like artifacts: OWL/DL axioms, domain and range restrictions, cardinalities, disjointness axioms, SHACL or ShEx shapes, SQL-checkable constraints, privacy-policy constraints, business rules, and KG quality reports. Many outputs remain partial. Ontology-learning systems often emit lightweight OWL axioms as a byproduct of D1 construction, while governance systems produce compliance checks, gap reports, or policy vocabularies rather than a complete integrity layer (Garza et al., 2024; Guo et al., 2026; Agarwal et al., 2025). We therefore code D2 as full only when the constructed artifact can be parsed or executed as a constraint and the evaluation treats it as such, as in NL-to-OWL axiom generation, OntoAxiom, SHACL or ShEx generation, Declare constraint extraction, and KG rule mining (Mateiu and Groza, 2023; Bakker et al., 2025; Westermann et al., 2025; Zhang et al., 2025; Park et al., 2026;

Luo et al., 2023).

Methodologically, the LLM is most often a translator, candidate generator, or pipeline component. Tier 1 systems propose local axioms or constraints from a bounded input span and leave schema alignment and acceptance to humans or external tools. Tier 2 systems sequence extraction, serialization, symbolic checking, and repair: OntoGenix follows this pattern for ontology construction (Val-Calvo et al., 2024), while compliance pipelines use LLM-based retrieval and comparison to relate policies to regulatory requirements (Garza et al., 2024; Guo et al., 2026; Agarwal et al., 2025). FAOS is the closest Tier 3 anchor because constraints participate in a broader agent-grounding architecture rather than remaining post-hoc validators (Luong Tuan and Sanyal, 2026). Ontology-to-tools and instruction-constraint systems make constraints executable, but their construction strategy remains fixed and is better read as Tier 2 support (Zhou et al., 2026; Sharma, 2026).

Evaluation is strongest when generated artifacts can be executed by symbolic validators. Existing evaluations include ontology expert review and structural metrics for generated OWL artifacts (Val-Calvo et al., 2024), query-equivalence or compilation checks for SHACL-to-SQL (Jakubowski and Van den Bussche, 2024), privacy or regulatory compliance verification (Garza et al., 2024; Guo et al., 2026), and KG quality metrics (Pellegrino et al., 2025). The common evaluation logic is *formal fitness*: does the generated or transformed constraint parse, execute, detect expected

violations, or improve an expert-judged compliance task? OntoAxiom adds axiom-level F1, AutomationML+SHACL and KG-repair work execute SHACL checks, NeOn-GPT runs syntax, reasoner, and OOPS! feedback loops, and ShEx generation introduces schema-specific metrics over large KGs (Bakker et al., 2025; Westermann et al., 2025; Lin et al., 2025; Fathallah et al., 2024; Zhang et al., 2025). The remaining weakness is scope: D2 is still treated as local validation rather than as a complete operational invariant layer tied to D1 entities, D3 preconditions, D4 governance rules, and D5 maintenance.

Action and process construction. The D3 corpus is smaller and more heterogeneous. Inputs include clean planning-domain descriptions, PDDL tasks, procedural text, API documentation, workflow descriptions, and enterprise-agent architecture descriptions. LLM-based PDDL domain and world-model generation demonstrates action-model construction with preconditions and effects (Guan et al., 2023; Gestrin et al., 2024; Oswald et al., 2024; Smirnov et al., 2024). FAOS provides a system-level anchor in which actions are constrained by role, domain, and interaction ontologies (Luong Tuan and Sanyal, 2026). The broader IE corpus contributes only indirectly: event and relation extraction methods can identify action-like mentions but usually do not produce executable process or tool specifications (Wei et al., 2023; Bi et al., 2024). More direct D3 work covers event and action extraction, procedural KG extraction, precondition and effect prediction, API operation extraction, and tool or function signature construction (Gao et al., 2023; Carriero et al., 2024; Xie et al., 2025; Lazar et al., 2025; Ni et al., 2025b,a; Patil et al., 2024).

D3 outputs describe what an agent can do and under which state conditions: PDDL action schemas, preconditions, effects, parameters, workflow graphs, BPMN-like process models, procedural DAGs, OpenAPI operation schemas, tool and function signatures, and state-transition structures. These artifacts differ from D1 outputs because they define operations rather than entity types, and from D2 outputs because constraints are embedded in executable dynamics. A precondition blocks an action, an effect changes the world state, a workflow gateway selects a branch, and an API schema defines a callable operation.

The dominant D3 method is fixed formalization

plus external checking. In planning work, the LLM proposes or revises an action model and a planner or validator exposes invalid predicates, missing preconditions, inconsistent effects, or unsolvable tasks (Silver et al., 2024; Guan et al., 2023; Gestrin et al., 2024; Oswald et al., 2024; Smirnov et al., 2024). In workflow and API settings, the analogous architecture segments the source document, asks the LLM to emit an intermediate representation, and then applies representation-specific checks such as process-model validation, simulation, conformance testing, or OpenAPI validation (Kourani et al., 2024, 2025; Garg et al., 2025; Ying et al., 2026; Lazar et al., 2024, 2025). Tier 3 would require more than emitting a process model: the system would need to decide which action specifications are missing, choose how to validate them, and revise the grounding layer so that actions remain aligned with entities, constraints, governance, and maintenance state. Current work offers only early evidence for this pattern (La Malfa et al., 2025).

D3 evaluation is naturally behavior-facing. Planning methods can test whether generated action models parse, produce valid plans, or solve benchmark tasks (Silver et al., 2024; Guan et al., 2023; Gestrin et al., 2024; Oswald et al., 2024; Smirnov et al., 2024). Workflow methods can be evaluated by structural similarity to expert process models, soundness, simulation, or conformance to event logs; API-specification methods can be checked for syntactic validity, operation coverage, and manual evidence alignment (Du et al., 2024; Kourani et al., 2025; Hörner et al., 2026; Garg et al., 2025; Lazar et al., 2025). This aligns with AOGC’s deployment goal: quality can be measured by whether an agent can choose, call, sequence, and constrain actions correctly, not only by whether a generated string resembles a reference annotation. The limitation is integration. Planning work assumes clean benchmark descriptions; process and API work targets workflows or tool contracts; and ontology-construction systems that cover D1 and partial D2 often omit executable actions entirely. The missing layer is the integration of ontology learning, process modeling, tool and API specification, and agent validation.

D Full Classification and Trends

Table 9 presents the coded evidence map used for our sample-level claims. The Scope column separates Core LLM construction methods from

Supporting representation or validator work and Adjacent analogues; only C rows are counted as LLM construction evidence in the main text. The dimension columns indicate which of the five functional dimensions each method addresses (F = Full, P = Partial, – = None) under the coding rules above. The Bib. column records bibliographic status so that accepted or archival rows are not conflated with public preprints or supporting standards.

Temporal and methodological trends. The coded map contains 32 core LLM construction methods, five supporting representation or validation works, and one adjacent enterprise-agent architecture. The bibliographic-status column records which rows were accepted or archival at the 21 May 2026 cutoff and which were public preprints, standards, or infrastructure records; these labels are used for claim calibration and are not weights in the counts. Among the 32 core rows, seven are Tier 1 subtask methods and twenty-five are Tier 2 fixed-pipeline methods; no core row satisfies our Tier 3 criterion of autonomously choosing missing construction activities, validation strategies, and maintenance actions. Contribution types are similarly skewed: eight core rows are benchmarks, nine are methods, and twelve are systems.

Dimension coverage mirrors the qualitative analysis. D1 is nearly ubiquitous: 31 of 32 core rows construct, evaluate, or assume entity, relation, or type structure, although only six make an integrated entity layer a full construction target. D2 is also common but narrower: 25 rows touch constraints or rules, with full coverage concentrated in local OWL/DL axioms, SHACL or ShEx shapes, Declare constraints, business rules, and KG rule mining. D3 appears in 15 rows, with nine full rows mostly in planning-domain, workflow and process, procedural-graph, and API-specification work. D4 appears in six rows, all partial, because they provide compliance verification, regulatory QA, or rule-extraction evidence rather than a complete enforceable governance layer. D5 appears in six rows, all partial: temporal validity, temporal KGQA, repair, and versioning work provide maintenance components but not full updates over $\langle V, C, A, G, M \rangle$.

The temporal trend is a shift from extraction-heavy D1 and D2 work in 2022–2024 toward D2 and D3 artifacts with executable validators in 2024–2026. Early LLM work largely asks whether prompted or instruction-tuned models can extract

terms, relations, triples, events, or candidate axioms. Later work increasingly wraps LLM generation with external checkers: OWL parsers, Hermit, OOPS!, SHACL or ShEx validators, planners, process-model soundness checks, simulators, and OpenAPI validators. This is methodologically important because operational grounding requires artifacts that parse and execute, not only annotations that resemble a reference.

Evaluation remains the main weakness. Core rows mainly use automatic extraction metrics, formal artifact checks, expert review, or task-specific utility such as planning success, compliance QA, or API-spec validation. These are valuable, but no coded core method combines the three conditions required for enterprise-readiness evidence: heterogeneous multi-source operational input, absence of a gold reference ontology, and downstream agent-behavior evaluation under incremental change. This is the empirical basis for our realism-gap claims.

E Retained Evidence Scope

The main text is compressed for length, so this appendix records the broader citation surface that informed the coding. The purpose is bibliographic continuity: the main text states the survey argument, while the appendix preserves the supporting evidence families.

Representation standards and formal foundations. The requirements framework is grounded in ontology, constraint, action, and workflow formalisms: OWL and SHACL standards (W3C OWL Working Group, 2012; Knublauch and Kontokostas, 2017), JSON Schema validation (Wright et al., 2022), rule languages such as SWRL (Horrocks et al., 2004), planning and hierarchical-task formalisms (McDermott et al., 1998; Erol et al., 1994), process and service descriptions (Object Management Group, 2014; Martin et al., 2004), and API contracts (OpenAPI Initiative, 2024).

Schema-level extraction and KG construction. The D1 evidence map includes work on formulation-sensitive relation extraction, retrieval-augmented demonstrations, recursive question answering, calibration, and LLM reranking (Zhang et al., 2023; Wan et al., 2023; Li et al., 2023b,a; Ma et al., 2023). It also includes instruction-following and schema-conditioned extraction systems (Wang et al., 2023; Gui et al., 2024; Zhou et al., 2024b;

Table 9: Coded evidence map. Scope: C=core LLM construction method, S=supporting representation/validator, A=adjacent analogue. Type abbreviations: Bench=benchmark, Meth=method, Rep=representation, Sys=system. Bib.: A=accepted/archival or peer-reviewed, P=public preprint at cutoff, S=standard/specification, T=tool or infrastructure. Evidence prefixes: Cstr=construction, Val=validator/executor, Use=downstream task use, Bench=reference/benchmark metric, Maint=maintenance/versioning, Rep=representation standardization.

Method	Scope	Tier	Type	Bib.	D1	D2	D3	D4	D5	Evidence
LLMs4OL (Babaei Giglou et al., 2023)	C	1	Bench	A	P	-	-	-	-	Bench: automatic metrics
SHACL-to-SQL (Jakubowski and Van den Bussche, 2024)	S	-	Meth	A	-	F	-	P	-	Val: SQL equivalence
DPV v2 (Pandit et al., 2024)	S	-	Rep	S	P	P	-	F	-	Rep: vocabulary coverage
Temporal validity (Soulard et al., 2025)	C	1	Bench	A	P	-	-	-	P	Bench: temporal accuracy
KGHeartBeat (Pellegrino et al., 2025)	S	-	Tool	T	P	P	-	-	P	Maint: quality metrics
NL-to-OWL axioms (Mateiu and Groza, 2023)	C	1	Meth	P	P	F	-	-	-	Cstr/Val: axiom conversion
OntoAxiom (Bakker et al., 2025)	C	1	Bench	P	P	F	-	-	-	Bench: axiom F1
Declare constraints (Park et al., 2026)	C	1	Bench	A	P	F	P	P	-	Bench: template accuracy
BREX/ExIde (Yang et al., 2025)	C	1	Bench	P	P	F	P	P	-	Bench: rule F1
PAGED (Du et al., 2024)	C	1	Bench	A	P	P	P	-	-	Bench: graph metrics
OntoGenix (Val-Calvo et al., 2024)	C	2	Sys	A	F	P	-	-	P	Use: expert evaluation
NeOn-GPT (Fathallah et al., 2024)	C	2	Sys	A	F	F	-	-	P	Val: reasoner/OOPS!
Ontogenia (Lippolis et al., 2025)	C	2	Sys	A	F	P	-	-	-	Val/Use: OOPS!/expert
ShEx schema generation (Zhang et al., 2025)	C	2	Meth	A	P	F	-	-	-	Bench: schema metrics
AutomationML+SHACL (Westermann et al., 2025)	C	2	Meth	A	P	F	-	P	-	Val: SHACL execution
SHACL repair (Lin et al., 2025)	C	2	Meth	P	-	F	-	-	P	Maint/Val: repair quality
ChatRule (Luo et al., 2023)	C	2	Meth	P	P	F	-	-	-	Use: rule utility/KGC
Capability ontologies (Vieira da Silva et al., 2024)	C	2	Meth	P	F	P	P	-	-	Val: OWL/SHACL checks
PrivComp-KG (Garza et al., 2024)	C	2	Sys	A	P	P	-	P	-	Use/Val: compliance check
ComplianceNLP (Guo et al., 2026)	C	2	Sys	A	P	P	-	P	P	Use: regulatory gap analysis
RAGulating Compliance (Agarwal et al., 2025)	C	2	Sys	A	P	P	-	P	-	Use: regulatory QA
OntoEditor (Hemid et al., 2024)	S	-	Tool	T	P	P	-	-	F	Maint: version workflow
Jelly-Patch (Sowinski et al., 2025)	S	-	Rep	A	-	-	-	-	F	Maint: RDF delta performance
TimeR ⁴ (Qian et al., 2024)	C	2	Meth	A	P	-	-	-	P	Use: temporal KGQA
PDDL planning with LLMs (Silver et al., 2024)	C	2	Meth	A	P	P	P	-	-	Use/Val: task debugging
World-model PDDL (Guan et al., 2023)	C	2	Sys	P	P	P	F	-	-	Use/Val: planning success
NL2Plan (Gestrin et al., 2024)	C	2	Sys	P	P	P	F	-	-	Val: planner validation
Planning-domain generation (Oswald et al., 2024)	C	2	Bench	A	P	P	F	-	-	Bench/Val: plan-set compare
Consistent PDDL (Smirnov et al., 2024)	C	2	Meth	P	P	P	F	-	-	Val: consistency checks
Process-model benchmark (Kourani et al., 2025)	C	2	Bench	A	P	P	F	-	-	Bench/Val: conformance
BPMNGen (Hörner et al., 2026)	C	2	Sys	A	P	-	F	-	-	Use: expert-quality study
text2flow (Ying et al., 2026)	C	2	Sys	A	P	P	F	-	-	Val: simulation feedback
SOPStruct (Garg et al., 2025)	C	2	Sys	P	P	P	F	-	-	Val: PDDL soundness
OASBuilder (Lazar et al., 2025)	C	2	Sys	A	P	P	F	-	-	Val: API-spec checks
AutoGraph-R1 (Tsang et al., 2025)	C	2	Meth	P	P	-	-	-	-	Use: RAG QA reward
AutoSchemaKG (Bai et al., 2025)	C	2	Sys	P	F	-	-	-	-	Cstr/Use: schema induction
Intention KG (Bai et al., 2026)	C	2	Sys	A	F	-	P	-	-	Use: intention/recommendation
FAOS (Luong Tuan and Sanyal, 2026)	A	-	Sys	P	F	P	P	P	P	Use: agent task study

Table 10: Realism audit for the 32 core rows. “Any” means the paper evaluates the property in at least one experiment; “full” means it is central to the evaluation.

Evaluation property	Any	Full	Typical evidence
Heterogeneous or multi-source input	7	2	API docs, policies, SOPs, KGs
No fixed gold ontology/schema reference	12	5	expert review, validators, utility
External formal validator or executor	17	12	reasoners, SHACL/ShEx, planners, OpenAPI checks
Downstream task or agent-behavior test	9	5	planning success, compliance QA, tool/API validation
Contradiction localization or resolution	0	0	not evaluated in core rows
Gap detection, abstention, or clarification	0	0	not evaluated in core rows
Incremental update after source change	3	0	temporal validity, repair, temporal KGQA
Production-scale continuous operation	0	0	not evaluated in core rows

Li et al., 2023c), coupled schema–instance construction and task-aware graph learning (Bai et al., 2025; Tsang et al., 2025), and behavior-grounded intention graphs (Bai et al., 2026), which motivate treating AOGC as schema-level structured generation rather than only ontology verbalization.

Events, procedures, actions, and tools. The D3 discussion draws on pre-LLM event-schema induction (Chambers and Jurafsky, 2008; Chambers, 2013; Cheung et al., 2013), LLM-based event and procedural extraction (Gao et al., 2023; Grohs et al., 2024; Carriero et al., 2024), precondition and effect prediction and PDDL translation (Xie et al., 2025; Liu et al., 2023a), tool-use and reasoning-action frameworks (Schick et al., 2023; Yao et al., 2023), procedural-graph and process-model studies (Du et al., 2024; Kourani et al., 2024; Li et al., 2025a), and more agentic PDDL construction (La Malfa et al., 2025).

Governance, validation, and ecosystem support. The D2–D4 boundary includes capability-ontology construction and validation (Vieira da Silva et al., 2024), SHACL-to-SQL compilation for enterprise-style validation (Jakubowski and Van den Bussche, 2024), and governance or institutional KG ecosystems (Geisler et al., 2025; Nuzzolese et al., 2025). These works support the claim that operational grounding must connect generated constraints to executable validators and organizational accountability, even when they are not themselves complete AOGC systems.

Maintenance, evolution, and agentic analogues. The D5 evidence includes continual KG learning and gradual evolution (Li et al., 2024, 2025b),

model and retrieval-based editing (Chen et al., 2025; Shi et al., 2024), temporal adaptation and history-aware reasoning (Hu et al., 2024; Xia et al., 2024), quality monitoring (Tuoizzo, 2025), and agentic skill or workflow evolution analogues (Liu et al., 2025, 2026; Wang et al., 2024; Zhang et al., 2026; Sharma, 2026; Zhou et al., 2026; Chi et al., 2026). We treat these as components or analogues unless they construct and maintain linked D1–D5 grounding artifacts.

F Search and Coding Protocol

The search protocol used the following query families, each combined with “LLM,” “large language model,” “GPT,” or “generative AI”: ontology construction, generation, and learning; schema extraction and induction; taxonomy extraction; relation extraction; axiom generation; description logic generation; SHACL and ShEx generation; constraint and business-rule extraction; PDDL and planning-domain generation; workflow and process-model extraction; BPMN generation; OpenAPI and tool-specification generation; API documentation mining; policy formalization; compliance knowledge graphs; regulatory rule extraction; ontology evolution; knowledge-graph versioning; and temporal validity. Venue scans covered ACL, EMNLP, Findings, NeurIPS, AAAI, ISWC, ESWC, ICAPS, K-CAP, BPM, software-engineering venues, and LLMs4OL proceedings from 2022 through May 2026.

Screening applied three inclusion tests: the paper had to use or evaluate an LLM in at least one construction, extraction, generation, validation, or maintenance stage; the output had to be schema-, action-, rule-, policy-, or maintenance-level structure rather than only instance facts; and the paper had to present a method, system, benchmark, representation, or substantive evaluation. Exclusion removed pure ontology usage for downstream tasks, instance-only KG population, non-LLM ontology construction (except as support evidence), and position papers without methodological or evaluative content.

Coding used the F/P/– criteria in Section D. Rows that do not themselves satisfy the LLM construction criterion but define important target representations, validators, or versioning mechanisms are marked **S** rather than **C**. Rows from agent or enterprise architectures that inform the AOGC target but do not autonomously construct D1–D5 artifacts

are marked **A**. Only **C** rows contribute to numerical claims in the main text.

Bibliographic status was coded separately from scope. A row marked **A** had an accepted, archival, or peer-reviewed conference or journal record available by the cutoff; **P** denotes a public preprint used as emerging or adjacent evidence; **S** denotes a standard, specification, or vocabulary whose authority is not a conference-acceptance claim; and **T** denotes tool or infrastructure evidence. These labels calibrate the evidential strength of boundary rows but do not change the 32/5/1 scope accounting.

The evidence prefix in Table 9 records the main relation between a row and its D1–D5 labels. *Cstr* means that the method constructs the artifact; *Val* means that a reasoner, validator, planner, simulator, compiler, or checker evaluates the artifact; *Use* means that the artifact is exercised in downstream QA, compliance, planning, tool use, or expert task evaluation; *Bench* means that coverage comes from a benchmark or reference metric; *Maint* means versioning, repair, or monitoring evidence; and *Rep* means a representation or vocabulary contribution. Mixed prefixes mark the reason for retaining a coarse F/P label when the same artifact is both constructed and validated or used.

G Tier Classification Examples

This appendix provides worked examples illustrating how the tier classification criteria resolve borderline cases.

LLMs4OL-style subtask benchmarks: Tier 1. Benchmarks that ask an LLM to perform term typing, taxonomy edge prediction, or relation extraction receive Tier 1 even when the subtask is central to ontology construction. The construction boundary is fixed by the benchmark designer, and the system does not decide which source to inspect next, which artifact family to construct, or how to integrate the result into a complete grounding layer.

NeOn-GPT-style fixed pipelines: Tier 2. A system receives Tier 2 when it sequences multiple construction steps such as competency-question generation, entity and relation extraction, OWL or RDF serialization, syntax checking, reasoner validation, OOPS! pitfall detection, and LLM-mediated repair. The key distinction from Tier 1 is that the architecture integrates several steps without a human manually invoking each one. It remains Tier 2 because the sequence and validation strategy are fixed

in advance.

PDDL or workflow generation with validators: Tier 2, not Tier 3. Planning-domain and process-model pipelines that translate text into PDDL, POWL, BPMN, or workflow graphs and then call planners, simulators, or soundness checkers are Tier 2 when the pipeline has a predetermined recipe. They may produce full D3 artifacts, but they do not become Tier 3 merely by iterating on validator feedback. Tier 3 would require the system to decide, for example, that missing policy constraints or entity distinctions must be constructed before the action model is reliable.

Agentic analogues: adjacent evidence, not core Tier 3. Systems such as FAOS, Voyager-style skill libraries, and autonomous code-repair agents are useful analogues for what future AOGC agents may need: persistent artifacts, execution feedback, self-revision, and task-level acceptance tests. We do not count them as full Tier 3 AOGC unless they autonomously construct and maintain linked D1–D5 grounding artifacts from domain sources. This conservative rule prevents the taxonomy from treating general agent autonomy as ontology-construction autonomy.

H Dimension Coverage Coding Examples

Dimension labels are assigned to the artifact that a method constructs, validates, or maintains, not to the topic that its prose happens to discuss. For Dimensions 2 and 3 this distinction is especially important, because many papers mention constraints, procedures, or explanations without producing a machine-interpretable constraint or action specification. The following examples document the decision rules used for borderline cases.

OWL axiom generation: D2 = F or P. A method receives D2 = F when its target artifact is a set of machine-interpretable integrity constraints or inference rules (for example, OWL/DL axioms, SHACL shapes, SWRL rules, or SQL or JSON-Schema constraints) and the paper evaluates those constraints as constraints. A focused OWL axiom-generation paper therefore counts as full D2 coverage when it asks the model to produce parseable axioms such as domain or range restrictions, disjointness, cardinality, equivalence, or subclass axioms, and then judges correctness by logical validity, reasoner checks, expert axiom review, or comparison to a gold axiom set. The tier may still be Tier 1 if the

LLM performs only this one predefined generation step.

The same surface form does not automatically imply $D2 = F$. We use $D2 = P$ when axioms appear as a narrow or auxiliary part of a broader ontology pipeline, or when the generated restrictions are limited to simple signatures and are not validated as a coherent constraint set. OntoGenix is coded this way in the classification table: it constructs OWL-oriented ontology drafts and includes axiom generation, but its main contribution is the end-to-end ontology-engineering pipeline rather than a dedicated, systematically validated constraint-generation method. In such cases, the method provides useful $D2$ material but does not demonstrate full integrity-rule construction.

Workflow-step extraction: $D3 = P$, not F . A paper that extracts ordered workflow steps from procedural text receives $D3 = P$ when it identifies actions or step order but stops short of an operational action model. Extracting a checklist such as “receive request,” “review documents,” and “approve request,” or a weak process graph over such steps, shows partial action and process understanding: the output describes what happens in a procedure and may support later action-schema construction.

It is not $D3 = F$ unless the output is specific enough for an agent, planner, or workflow engine to execute or validate. Full $D3$ requires structured operation definitions such as typed parameters, preconditions, effects, state transitions, API or tool schemas, BPMN execution semantics, HTN decompositions, or PDDL operators. Step extraction without those elements leaves the hardest grounding decisions to a human or downstream system: what objects the action consumes, what state must hold before it runs, what state changes afterward, and which failures are invalid. We therefore treat workflow-step extraction as evidence of $D3$ relevance, but not as full action specification.

PDDL action generation: $D3 = F$. PDDL action generation receives $D3 = F$ when the method outputs action schemas with the formal components required by planning systems: action names, typed parameters, preconditions, and effects. These artifacts directly encode what actions are available and how they transform world state, which is the core of Dimension 3. Unlike a natural-language procedure summary, a PDDL operator can be parsed by a planner and tested through plan generation, simulation, or execution-style validation. A method

can therefore receive full $D3$ coverage even when it does not address the rest of the operational grounding layer: object-type coverage may be only $D1 = P$, governance may be $D4 = -$, and temporal maintenance may be $D5 = -$, but the action specification itself is complete enough to count as $D3 = F$.

LLM explanation-only work: $D2$ and $D3 = -$. An LLM paper that only explains an existing ontology, verbalizes a class or property, summarizes why a rule might be useful, or provides a rationale for a human designer receives $D2 = -$ and $D3 = -$ unless it also emits or validates a machine-interpretable artifact for those dimensions. Natural-language explanations can be valuable for usability, documentation, or expert review, but they do not themselves define an integrity constraint, inference rule, action schema, workflow model, or executable procedure. Under our coding rules, coverage requires constructed structure, not merely explanatory text about structure.